

Supplementary Material

Formalization Scope and Explicit-Example Checks

Myunglae Jo

S1. Purpose

The companion source package verifies selected finite combinatorial components of *Small Rotation Blocks and the Order-Six Stable Marriage Problem*. It is not an end-to-end Lean formalization of stable marriage or of the statement $f(6) = 48$. The manuscript supplies the complete upper-bound argument and cites the published lower-bound construction.

S2. Fixed-token combinatorics

The module `F6SmallBlockTokenCapacity.lean` uses fixed finite types of participants B and tokens W , bijective states $B \simeq W$, supports of size at least two, unchanged nonparticipants, and strictly worsening participants. Its principal declarations are

```
TokenRun.participation_count
TokenRun.TWO_SITE_CAPACITY
TokenRun.THREE_SITE_CAPACITY
TokenRun.SMALL_BLOCK_TOKEN_CAPACITIES
TokenRun.SIX_SITE_INCIDENCE_OPTIONAL
```

in namespace `F6SmallBlockTokenCapacity`. Writing q for the number of transitions in a run, they prove the participation bound $|B| - 1$, the capacities $q \leq 1$ for $|B| \leq 2$ and $q \leq 2$ for $|B| \leq 3$, and $q \leq 15$ for $|B| \leq 6$. The three-participant proof includes incidence equality, forced pair swaps, and token return.

S3. Abstract support geometry

The module `F6RotationSupportCI2.lean` has an abstract finite poset R and a support map to a finite set of at most six participants. It assumes support size at least two and comparability of elements whose supports intersect. The capacities for convex subsets using at most two and three participants are the explicit hypotheses `HCAP2` and `HCAP3`. From these hypotheses, the declaration

```
F6RotationSupport.widthLE3_and_ci2_of_rotation_support
```

proves width at most three and `CI2`. The source package does not construct actual stable-marriage rotations or formally connect their execution to the abstract token model; that connection is made in the manuscript.

S4. Finite-poset counting

The modules `F6ConvexWindow.lean`, `F6ConvexWindowFull.lean`, and `F6ConvexWindowN15.lean` establish the pair and triple estimates and the fifteen-element refinement. The final declaration is

```
F6ConvexWindow.totalAntichains_le_48
```

with hypotheses `WidthLE 3`, `CI2`, and `Fintype.card V <= 15`. No preference-profile classification is an input to this theorem. The chain decomposition is supplied by the bundled Lean proof of Dilworth’s theorem.

S5. Dilworth source and reproducibility

The bundled `DilworthTheorem.lean` has source repository

```
https://github.com/sneed-and-feed/lean-theorems-1
```

with frozen commit

```
4fce2c92ad4321064a06afdc86d0272bc625ce14
```

and SHA-256

```
bfb7c9505c0bdc4d8ba7ff49b3a1b39243ee35f3ad21d87a287a3df3a48faea
```

The package records the CC0-1.0 dedication associated with that source repository.

The frozen v07 `LEAN_FORMALIZATION_REVIEWER_PACKAGE.zip` has SHA-256

```
0d6524cf50fa8dfe34c456ebef1476add48e4e39bbd00ec37938a2f4b912fa63
```

It pins Lean 4.27.0 and mathlib v4.27.0; the mathlib commit is

```
a3a10db0e9d66acbebf76c5e6a135066525ac900.
```

The public source and reproducibility repository is <https://github.com/insamrael/stable-marriage-f6>.

In that repository, the audited source files are exposed directly under `lean/` and migrated to a Palomar-supported Lean toolchain, with the toolchain, Mathlib revision, and transitive dependencies pinned by `lean-toolchain`, `lakefile.toml`, and the committed `lake-manifest.json`. From the public-repository root, run

```
lake build
```

```
lake env lean palomar/AxiomAudit.lean
```

The v08 release audit records the build and axiom outputs and separately checks the migrated source against the frozen v07 source. The reported load-bearing axioms are `propxt`, `Classical.choice`, and `Quot.sound`.

S6. Components outside the formalization

The restricted-instance observation, the explicit extremal fifteen-element poset, the necessary equality conditions, the DS_3-P_* isomorphism, the general order- N support-budget statement, and the stable-matching-downset correspondence remain outside the bundled formalization. The upper bound does not rely on the auxiliary example calculations below.

S7. An explicit extremal poset

For $P_* = \{1, \dots, 5\} \times \{1, 2, 3\}$, put

$$(i, a) < (j, b) \iff j \geq i + 2 \text{ or } (j = i + 1 \text{ and } a \neq b).$$

The manuscript directly proves that P_* has width three and CI2.

Its antichain counts are $(1, 15, 27, 5)$, totaling 48. An auxiliary check enumerated all 2^{15} subsets, tested pairwise incomparability, and tested CI2 for every comparable pair. It returned the same counts, no larger antichains, and maximum common-incomparability size two. This independently checks the explicit extremal example.

S8. A classical order-six lower-bound instance

The preference instance used by the auxiliary checker is the DS_3 construction of Benjamin, Converse, and Krieger [1]: DS_n is defined in their Section 3 (p. 287) as a $2n \times 2n$ Latin square built from S_n , with an algebraic definition of its entries; Fig. 1 on that page displays DS_3 explicitly, and Section 5 (p. 291) states that DS_3 produces 48 stable matchings. The table below is a transcription of Fig. 1 and the Section 3 algebraic definition. The publication identifies and defines the instance; the checker verifies the mathematical properties of the transcribed input. The row indexed by a man gives his rank of each woman, with rank 1 best:

$$R = \begin{pmatrix} 1 & 3 & 5 & 2 & 4 & 6 \\ 3 & 5 & 1 & 6 & 2 & 4 \\ 5 & 1 & 3 & 4 & 6 & 2 \\ 2 & 4 & 6 & 1 & 3 & 5 \\ 6 & 2 & 4 & 3 & 5 & 1 \\ 4 & 6 & 2 & 5 & 1 & 3 \end{pmatrix}.$$

Woman j assigns rank $7 - R_{ij}$ to man i . Enumerating all $6! = 720$ permutations p and rejecting any one for which there is a pair (i, j) satisfying

$$R_{ij} < R_{i,p(i)} \quad \text{and} \quad 7 - R_{ij} < 7 - R_{p^{-1}(j),j}$$

returns 48 stable matchings. The bundled checker also recovered 15 join-irreducibles of the complete stable-matching lattice, with antichain counts $(1, 15, 27, 5)$, and found all 36 pairs attainable. All 15 join-irreducible cover moves have two-man support, and each man participates five times.

S8.1. Explicit isomorphism certificate

Let $r_1, \dots, r_{15}$ be the join-irreducibles in the deterministic order of the bundled checker: stable matchings are sorted by the sum of men's ranks, then lexicographically, and those with exactly one lower cover are retained in that order. Define

$$\phi(r_{3(i-1)+k}) = \begin{cases} (i, k), & i \text{ odd}, \\ (i, 4 - k), & i \text{ even}, \end{cases} \quad 1 \leq i \leq 5, \quad 1 \leq k \leq 3.$$

The five levels, written as indices, are

$$\{1, 2, 3\}, \quad \{4, 5, 6\}, \quad \{7, 8, 9\}, \quad \{10, 11, 12\}, \quad \{13, 14, 15\}.$$

Covers occur only between consecutive levels. Each such cover graph is $K_{3,3}$ minus a perfect matching; the four missing matchings are

$$\begin{aligned} &\{(1, 6), (2, 5), (3, 4)\}, \\ &\{(4, 9), (5, 8), (6, 7)\}, \\ &\{(7, 12), (8, 11), (9, 10)\}, \\ &\{(10, 15), (11, 14), (12, 13)\}. \end{aligned}$$

The checker verifies for every pair u, v that

$$r_u \leq r_v \iff \phi(r_u) \leq_{P_*} \phi(r_v).$$

Writing $J(DS_3)$ for the join-irreducible poset of the stable-matching lattice, this gives $J(DS_3) \cong P_*$. For a finite poset R , the join-irreducible elements of its ideal lattice $\mathcal{J}(R)$ are precisely the principal ideals $\downarrow r$, so $r \mapsto \downarrow r$ identifies R with $\text{Ji}(\mathcal{J}(R))$. Combining this identification with the classical correspondence between the stable-matching lattice and $\mathcal{J}(R)$, where R is the rotation poset (see the Irving–Leather reference in the manuscript), shows that this join-irreducible poset is isomorphic to the rotation poset of DS_3 . This finite certificate is auxiliary to the universal upper-bound proof.

S9. Reproducing the auxiliary checks

The supplementary package includes `AUXILIARY_CHECKS/`, containing `check_ds3_pstar.py`, its expected `DS3_PSTAR_CHECK.json`, a README, and SHA-256 checksums. From that directory run

```
python3 check_ds3_pstar.py
```

The checker uses only the Python standard library. The command regenerates `DS3_PSTAR_CHECK.json` and must reproduce the bundled expected JSON exactly, byte-for-byte; its expected hash is recorded in the README and `SHA256SUMS.txt`.

References

- [1] A. T. Benjamin, C. Converse, and H. A. Krieger, How do I marry thee? Let me count the ways!, *Discrete Applied Mathematics* 59 (1995), 285–292. doi:10.1016/0166-218X(95)80006-P.